DevOps STAR Scenarios

These examples turn the five lab incidents into realistic production DevOps narratives. Each example follows the STAR flow naturally without labeling its individual parts.

1. Bad Container Image Deployment

One of our applications was running on Amazon EKS when a release created new pods that could not start, while the existing version continued serving traffic. I needed to determine why the rollout was blocked and recover it without disrupting the healthy pods. The rollout status showed fewer updated replicas than desired, and Kubernetes warning events identified the backend Deployment as the affected workload. Kubernetes showed `ImagePullBackOff`, and `kubectl describe pod` confirmed that the cluster could not retrieve the configured image. I compared the Deployment image with the tags available in its Amazon ECR repository and found that the requested tag did not exist. I rolled the Deployment back to its known working revision, monitored it until the new pods became Ready, and verified the application. Image-tag validation was then added to the delivery process.

Natural spoken version

"One of our applications was running on EKS when a release produced new pods that would not start, although the existing version still served traffic. The rollout was stuck and Kubernetes reported `ImagePullBackOff`. Pod details and events showed that the cluster could not pull the configured image. I compared the Deployment tag with the tags in ECR and found that it did not exist. I rolled back to the working revision, monitored the rollout until the pods became Ready, and tested the application. We then added image-tag validation before deployment."

2. Bad ConfigMap Rollout

An internal application was running on Amazon EKS when a configuration rollout caused new backend pods to stop becoming Ready while existing pods continued serving traffic. I needed to determine what changed and why the problem affected only the new pods. Kubernetes showed unavailable replicas, increasing restarts, and probe-failure events. I checked the rollout, pod events, and application logs. The process started successfully, but the logs reported that it was listening on port 4001 while the readiness and liveness probes still checked port 4000. I compared the live ConfigMap with the Deployment probes, restored `PORT` to 4000, restarted the Deployment, and monitored it until all pods became Ready. External traffic succeeded and restarts stopped. Configuration and health-probe validation was added to the deployment process.

Natural spoken version

"After a configuration change to one of our internal applications, new EKS pods stopped becoming Ready while existing pods kept serving. Kubernetes showed unavailable replicas, restarts, and probe failures. The application logs showed that the process was listening on 4001, but Kubernetes was still checking 4000. I

compared the ConfigMap with the probe configuration, restored the correct port, restarted the Deployment, and watched the rollout complete. I then tested the application and confirmed that the restarts stopped."

3. OOMKilled and Resource Limits

An internal backend service on Amazon EKS became unstable because pods repeatedly restarted. I needed to determine whether the restarts came from application behavior, Kubernetes configuration, or the underlying infrastructure. Kubernetes pod details showed `OOMKilled` and exit code 137 in the previous container state. I compared the memory the healthy pods were using in `kubectl top` with the Deployment request and limit and found that the limit had been reduced below what the Node.js process required. I restored the measured resource baseline, monitored the rollout, and verified that pods remained Ready, restart counts stopped increasing, and memory stayed below the limit. Resource values for similar workloads were then reviewed using observed utilization rather than arbitrary limits.

Natural spoken version

"One of our internal backend services on EKS became unstable because pods kept restarting. Kubernetes reported `OOMKilled`, so I compared actual usage from `kubectl top` with the configured request and limit. The limit had been reduced below what the process needed. I restored the measured resource baseline, monitored the rollout, and confirmed stable pods, no new restarts, and normal memory usage."

4. ALB and Kubernetes Ingress Routing

One of our applications on Amazon EKS had healthy workloads, but users received HTTP 404 responses from its public URL. I needed to locate the failure across Route 53, the Application Load Balancer, Kubernetes Ingress, Services, endpoints, and pods. Kubernetes reported healthy frontend and backend pods, DNS resolved correctly, and the ALB accepted connections, yet `/` returned 404 while `/api` paths still answered. I reviewed the Ingress and found that the public route pointed to the backend Service on port 4000 instead of the frontend Service on port 80. I restored the frontend Service as the Ingress backend, waited for the AWS Load Balancer Controller to reconcile, and tested the application externally. Traffic-path validation was added to deployment checks.

Natural spoken version

"Users received 404 responses from one of our applications even though the EKS workloads were healthy. The API paths still worked, but the home page did not. I traced the request through Route 53, the ALB, Ingress, Services, endpoints, and pods. The Ingress pointed to the backend instead of the frontend. I restored the frontend route, waited for the controller to reconcile, and confirmed the public URL worked again."

5. High Application Latency

Users experienced increased response times from one of our applications running on Amazon EKS. I needed to determine whether the latency came from AWS infrastructure, Kubernetes, or a particular workload before making any changes. Timing requests with `curl` showed that API calls took about three seconds while the

static frontend stayed fast, which pointed at the backend. Kubernetes CPU, memory, readiness, replicas, and restart counts remained near baseline. Port-forwarding to individual backend pods showed which processes were slow, and their chaos state revealed that a response-delay setting was enabled in the container. I cleared the setting from each affected process, generated application traffic, and confirmed that response times returned to baseline and pods stayed healthy.

Natural spoken version

"Users reported slower responses from one of our applications, so I measured before restarting anything. API calls took about three seconds while the frontend was fast, so the issue was in the backend. Kubernetes CPU, memory, readiness, and restart counts were normal, which narrowed it to workload behavior. I found a response-delay setting enabled in the container, cleared it from the affected processes, generated traffic, and confirmed normal response times and healthy pods."